

Aircraft Data Visualization

Review of Version Two / HCI584 / Andy Walker

Project Introduction

In the past, radar research has been limited to large companies and those with exceptional budgets. However, Amateur Radio enthusiasts are increasingly able to buy and build capable microwave systems for communications, sharing of data, and sensing of obstacles. These capabilities are primed to push amateur electronics into the realm of real microwave sensors like radars. Radars use radio signals to detect and track 'targets' in their field of view, and larger targets make for easier, more certain detections. Commercial aircraft provide excellent targets of opportunity for such a system. They are physically and electrically large, they move at high speed and broadcast their position regularly via radio links that can be used to provide a source of truth for the hobbyist measurement. This enables educational radar training by providing a test bed for unclassified, amateur radio compatible radar. This project captures the broadcast position data and maps it relative to the sensor location to provide 'targets of opportunity' to the sensor.

How to install

First, install python. Development was with 3.13 but no dependencies required older packages and it is expected that newer Python should work fine. Install the unique requirements by opening a terminal in the folder where the files were downloaded and executing:

```
``pip install -r requirements.txt``
```

This will install the non-standard libraries that this project requires. Keys are not required but do make the ADSB API data pulls much more responsive. More can be found on getting ADSB API keys at <https://rapidapi.com/adsbx/api/adsbexchange-com1>. Instructions on using the API are detailed later.

Once finished successfully, the final version should be started with:

```
``streamlit run .\main.py``
```

This should automatically open a web browser window that looks like Figure 1 and Figure 2.

How to use

In the Streamlit user interface, the primary map interface is front and center. The map provides range rings that illustrate the range at which the nominal sensor should detect targets of 10dB square

meters (Green), 20dB square meters (Yellow), and 30dB square meters (Red). For reference, a 10dB-SM target is a typical commuter aircraft, depending on the angle it presents to the sensor. A 20 dB-SM target is a very large aircraft like a 747 or B-52, again depending on the angle to the sensor. Finally, a 30 dB-SM target is the size of a battleship, impossibly large for airborne targets but possible for some intentional targets like corner reflectors.

To the side, buttons are present to configure range details of the aircraft plotter (1), the automatic map updating (2), start and stop the background application (3), and the log of the background process (4). In (1), the latitude, longitude, and altitude mark the location of the sensor (the nominal radar sensor), and the range number sets how far out the presumed sensor can sense. In practical terms, this is used to limit the number of aircraft plotted to improve update rate performance and memory utilization.

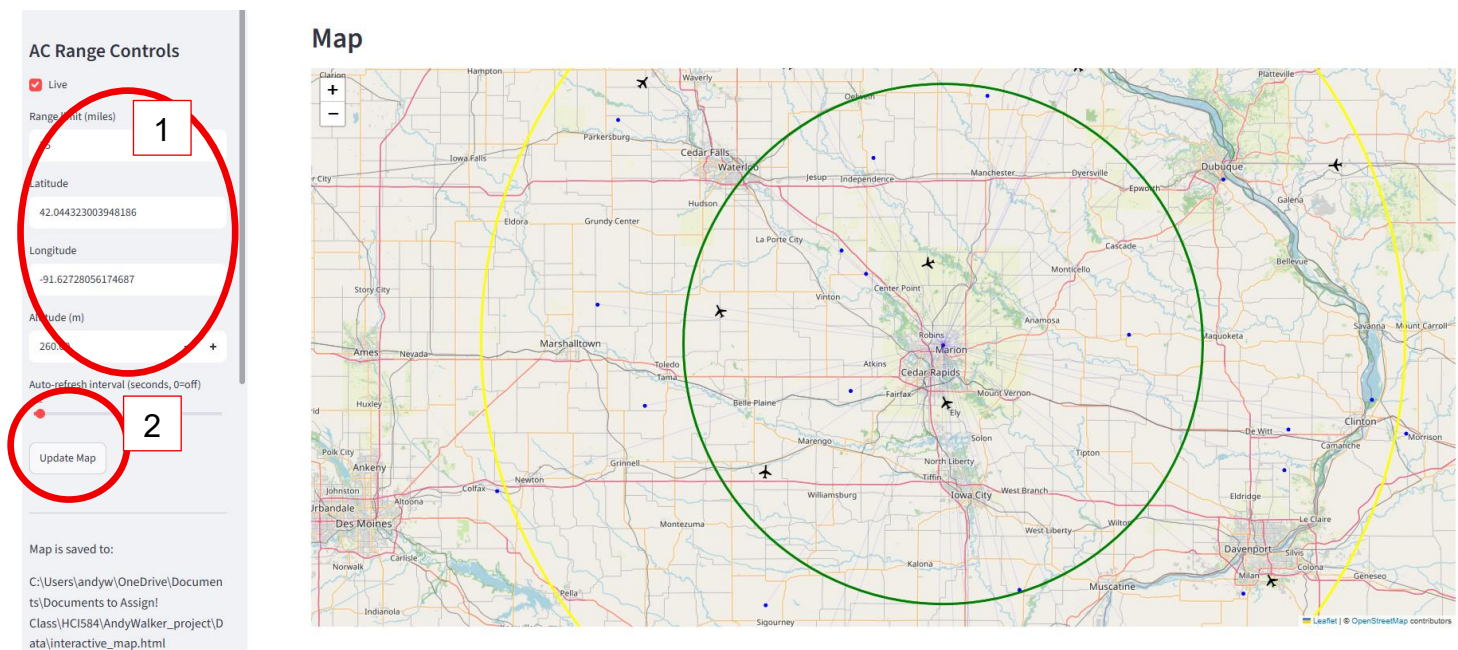


Figure 1: HTML Interface Hosted by Streamlit

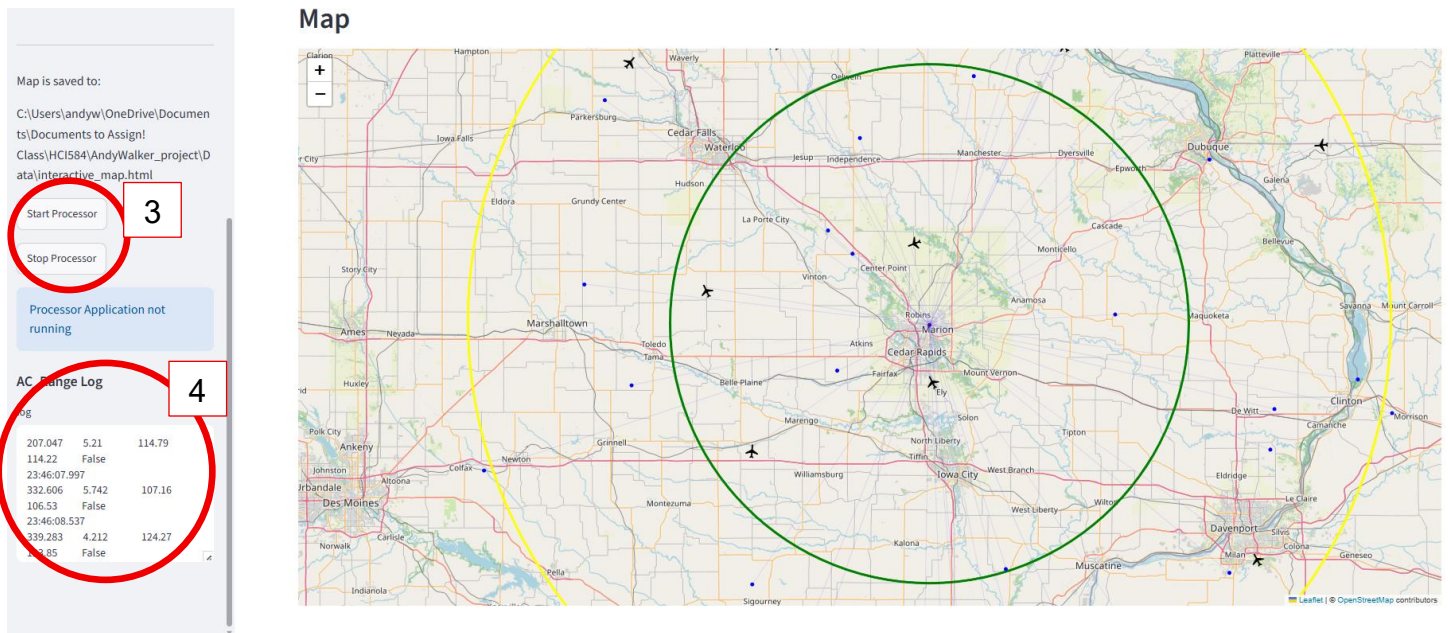


Figure 2: HTML Interface with Side Panel Extended

APIs Used

To plot this date, 2 APIs are used. The first provides the ADSB data of the aircraft in the area, and the second provides topographic maps from which terrain masking is extracted.

<https://globe.adsbexchange.com/>

ADS-B Exchange is the world's largest independent and unfiltered global flight tracking data network, providing unmitigated civil and military flight tracking data. It operates via a crowdsourced community of feeders sharing raw receiver data without filtering out specific aircraft. The platform focuses on raw data access without FAA-compliance filtering or requested blockings for military and private aircraft provided by global volunteers hosting hardware receivers feeding real-time positional data (latitude, longitude, altitude, and velocity) into the collective map.

This API can be used with or without a key. However, responsiveness is greatly improved via a low-cost API key subscription. This key should be placed in the 'keys.py' file in the root directory of the project in "double quotes". That key is then referenced in main.py via the

```
``from keys import access_token as api_key``
```

reference.

<https://open-meteo.com/en/featuresa>

OpenMeteo provides a wide range of weather-related data via an easy-to-use API. Among the interfaces provided is an Elevation API providing access to a terrain database that returns the terrain elevation at the provided coordinates. Terrain elevation is provided in meters for any set of WGS84

coordinates using a 90 m resolution digital elevation model and accepts batches of up to 100 coordinate pairs per request.

This API requires no key to extract elevation information. The returned data is limited and does impact update rate.

Developers Adendum:

Additional details are available in the “Developers_Addendum.md” file in the /Doc folder